INT161 · Basic Backend Development

Pagination and DTO

Return big lists a page at a time with page, limit and offset, and shape responses with a DTO.

Why Pagination | page, limit, offset | LIMIT, OFFSET and COUNT | Response Structure

Data Transfer Object | Mapper and apiResponse | Glossary

01

Why Pagination

Take an online shop with a hundred million products. If the **back end** loaded them all and sent them to the **front end**, it would not survive. So lists come in pages, and the front end asks for the page it wants.

- Big **responses** fill the **server** memory, load the database, and cannot cross the network quickly.
- Pages give the **client** a predictable size. The front end can also guess a sensible **page size**.
- The front end can only **paginate** if the back end supports it.
- There are two approaches: offset-based, which we do, and **cursor-based**. **Cursor** is quicker for very deep pages, but it only moves through pages in order, so you cannot jump to page 500. **Offset** lets you jump anywhere, so we use it.

02

page, limit, offset

The client sends two **query** parameters, for example GET /api/products?page=2&**limit**=20.

- page: which page the client wants. limit: how many records per page.

- The back end works out offset: how many records to skip so the right page starts. offset = (page - 1) x limit.

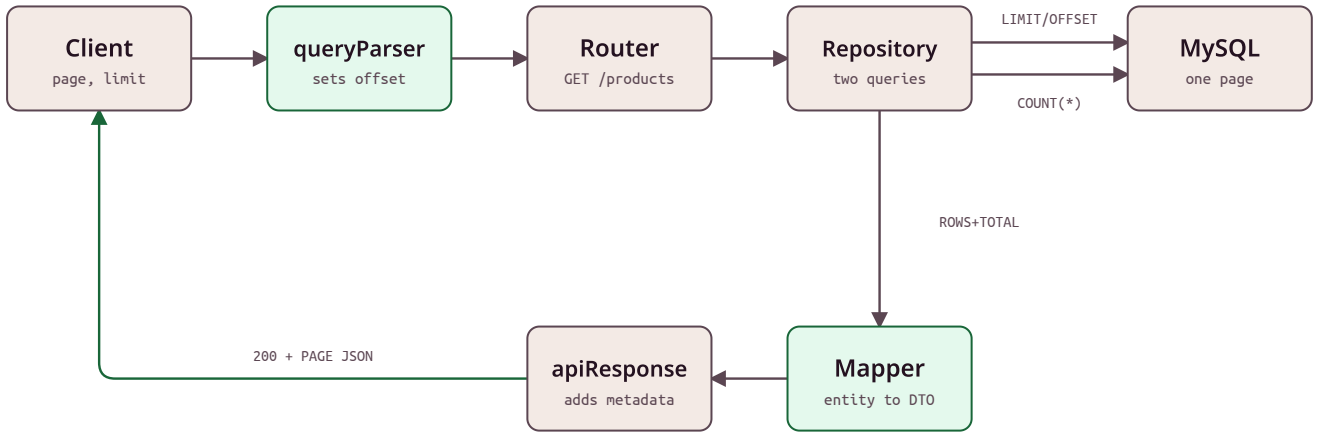
page = 2 (the client sent this)

limit 10 **20** 50 500

138 items in total. The server uses page = 2 and limit = 20.

offset	**SQL**	**items**
`(2 - 1) x 20 = 20`	`LIMIT 20 OFFSET 20`	`21 to 40`

totalPages	**hasNextPage**	**hasPrevPage**
`ceil(138 / 20) = 7`	`true`	`true`



- Validate the input. Page is at least 1, so 0 or a negative becomes 1. If limit is missing use a default of 20. Cap limit at 100 so nobody asks for a huge page.
- Read page and limit in one **middleware**, not in every **router**.

pagination-middleware.js

```
// middlewares/pagination-middleware.js
export const queryParser = () => (req, res, next) => {
  const query = req.query || {};
  if (query.page == undefined || query.limit == undefined) {
    req.pagination = null; // the client did not ask for pages
    return next();
  }
  const page = Math.max(parseInt(query.page), 1); // never below 1
  const limit = Math.min(parseInt(query.limit), 100); // never above 100
  const offset = (page - 1) * limit;
  req.pagination = { page, limit, offset };
  next();
};
```

app.js

```
import { queryParser } from './middlewares/pagination-middleware.js';
const app = express();
app.use(express.json());
app.use(queryParser()); // before the routers
```

> ⚠ **Order**
>
> app.use(queryParser()) must come before the routers. The routers read req.**pagination**, and it does not exist yet if the middleware runs after them.

03

LIMIT, OFFSET and COUNT

Two queries. One fetches the page, one counts everything, so the response can say how many pages exist.

repositories/product-repository.js

```
const COUNT_ALL = `SELECT COUNT(*) as total FROM ${TABLE_NAME}`;
const FIND_ALL_SQL = `SELECT * FROM ${TABLE_NAME}`;
async function count() {
  const [[{ total }]] = await db.query(COUNT_ALL);
  return total;
}
export async function findAll(pagination) {
  if (!pagination) {
    const [rows] = await db.query(FIND_ALL_SQL);
    return rows;
  }
  const SQL = FIND_ALL_SQL + ' LIMIT ? OFFSET ?';
  const { limit, offset } = pagination;
  const [rows] = await db.query(SQL, [limit, offset]);
  rows.totalItems = await count(); // the layers above need it for the metadata
  return rows;
}
```

- LIMIT and OFFSET are **placeholders** too. Values go in the array: limit first, offset second.
- No pagination object means the client did not ask, so return everything. That is fine for short lists, such as a list of provinces. A big table must never do this.
- Always use **prepared statements**. This also guards against **SQL injection**.
- If you sort or filter on a column, give it a **database index**. Without one, a big table gets slow.
- The cost of offset: to start at record 10,000 the database still reads past the first 10,000. That is why very deep pages are slow, and **cursor pagination** exists.

04

Response Structure

The front end needs more than data. It needs to know where it is: which page, how many in total, whether there is a next or previous page. That drives the page buttons.

Response with pages

```
{
  "success": true,
  "data": [ ... ],
  "pagination": {
    "total": 138,
    "page": 2,
    "limit": 20,
    "totalPages": 7,
    "hasNextPage": true,
    "hasPrevPage": true
  },
  "timestamp": "2026-10-08T15:00:00.000Z"
}
```

- hasPrevPage false means there is no previous page, so the front end disables Previous. hasNextPage works the same way for Next.
- Keep one shape for every **endpoint**: success, data, and an optional pagination object. The slide lists totalItems and hasPreviousPage in one outline, and the code uses total and hasPrevPage. The exam spec uses the code names.

05

Data Transfer Object

A table is an **entity**, and it carries every column. Some columns are only for the database, like last update. Some the front end does not need. Some must never leave, like a password hash. A **DTO** is the shaped copy you send instead.

Send the entity | Send a DTO

```
{
  "id": ...,
  "name": ...,
  "email": ...,
  "role": ...
}
```

password_hash and created_at never leave the server. The shape is the same on every endpoint. Even a hashed password should not be sent, because a hash can be attacked by brute force.

- It keeps the **API** consistent, and separates the API from the database model.
- It can rename fields. JavaScript and **JSON** usually use **camelCase**, but a database column may be **snake_case**. The back end converts.
- It can combine entities: a customer with their orders in one response, so the front end makes one call instead of two.
- It works both ways. For a **request**, accept only the fields you need, not every column of the entity.

Summary

A few fields, for a list. A product list needs the name and the price, not the long description.

Detail

Everything, for one item. It is shown when the user clicks into a product.

Where to map

Do it in the **service layer** or the **controller**, either works. JavaScript makes it easy to drop or add fields. Even so, put the logic in a reusable class instead of writing it inline on every endpoint.

06

Mapper and apiResponse

One **mapper** per entity. A product gets a product mapper, a customer gets a customer mapper. They are not one general class.

mappers/product-mapper.js

```
// mappers/product-mapper.js
export default class ProductMapper {
  static toSummaryDTO(entity) {
    return {
      id: entity.productCode,
      name: entity.productName,
      scale: entity.productScale,
      retailPrice: entity.MSRP,
    };
  }
  static toDetailDTO(entity) {
    const { MSRP, buyPrice, ...data } = entity; // pull out the two we rename
    return { ...data, retailPrice: MSRP, price: buyPrice };
  }
  static map(entity, viewType) {
    return viewType === 'full' ? this.toDetailDTO(entity) : this.toSummaryDTO(entity);
  }
  static maps(entities, viewType) {
    const { totalItems } = entities;
    const dtos = entities.map((e) => this.map(e, viewType));
    if (totalItems) dtos.totalItems = totalItems;
    return dtos;
  }
}
```

- The client chooses the view with a query parameter, ?view=full. The default for a list is **summary**. The default for one item is full.
- maps loops over the entities and keeps the totalItems that the **repository** attached.

The last step is one function that builds the whole JSON, so every endpoint looks the same.

mappers/api-response.js

```
// mappers/api-response.js
import httpStatus from 'http-status';
export async function apiResponse(res, data, pagination = null, status = httpStatus.OK) {
  if (!pagination) {
    return res.status(status).json({ success: true, data, timestamp: new Date().toISOString() });
  }
  const { limit, page } = pagination;
  const total = data.totalItems;
  return res.status(status).json({
    success: true,
    data,
    pagination: {
      total, page, limit,
      totalPages: Math.ceil(total / limit),
      hasNextPage: page * limit < total,
      hasPrevPage: page > 1,
    },
    timestamp: new Date().toISOString(),
  });
}
```

routes/product-route.js

```
// routes/product-route.js
router.get("/", async (req, res) => {
  const pagination = req.pagination;
  const viewType = req.query.view || "summary";
  const objects = await service.getAllObjects(pagination, viewType);
  apiResponse(res, objects, pagination);
});
router.get("/:id", async (req, res) => {
  const viewType = req.query.view || "full";
  const object = await service.getObject(req.params.id, viewType);
  apiResponse(res, object);
});
```

- The router does not build JSON itself. It hands the result and the pagination to apiResponse, which writes the response with or without page **metadata**.
- Because everything goes through that one function, the JSON is guaranteed to be consistent.
- Check it in the **browser** or **Postman**: page 3 with limit 3 shows hasNextPage and hasPrevPage true. Ask for a bigger limit and the number of pages drops.

07

Glossary

API
: Application Programming Interface: the doors a program offers to other programs.

back end
: The part on the server: it handles data, rules and the database, and answers the front end with JSON.

browser
: The client most people use, such as Chrome, Edge or Safari.

camelCase
: Words joined with capitals, like userName. The convention for JavaScript variables.

client
: The side that asks. Usually a browser, but it can be any program or device.

controller
: The layer that receives the HTTP request and sends the response. It delegates the real work.

Cursor
: Instead of skipping, remember where the last page ended and continue from there. Fast for deep pages, but you cannot jump to page 500.

cursor pagination
: Instead of skipping, remember where the last page ended and continue from there. Fast for deep pages, but you cannot jump to page 500.

cursor-based
: Instead of skipping, remember where the last page ended and continue from there. Fast for deep pages, but you cannot jump to page 500.

database index

A structure the database keeps to find rows fast. Index the columns you sort or filter by.

DTO

A shaped copy of an entity for the API: only the fields the client needs, in the format you agreed.

endpoint

One method plus path an API answers, such as GET /api/subjects.

entity

A database record as stored: a row of a table, with every column.

front end

The part users see and touch. It runs in the browser or on the phone.

JSON

JavaScript Object Notation: a plain text format for data that both humans and programs can read.

limit

How many records one page holds. The client sends it as limit, and the server caps it.

mapper

A class that turns an entity into a DTO, so the conversion lives in one reusable place.

metadata

Data about the data. In a page response: the page number, limit, total items and total pages.

middleware

A function (req, res, next) that runs between the request and the route handler. It can change the request, answer it, or pass it on.

Offset

How many records to skip before the page starts: (page - 1) x limit.

page size

How many records one page holds. The client sends it as limit, and the server caps it.

paginate

Returning a big list one page at a time, so responses stay small and fast.

pagination

Returning a big list one page at a time, so responses stay small and fast.

placeholders

The ? in a prepared statement. Values fill the ? in order.

Postman

A tool for sending any HTTP method to your API, so you can test it before the front end exists.

prepared statements

SQL written with ? where the values go, and the values sent separately. The database never mistakes a value for a command.

query

The part of the URL after ?, like page=2&size=10. Used to filter or page a list.

repository

The layer that reads and writes the database. No business rules, no HTTP.

request

What the client sends: method, URL, headers and sometimes a body.

responses

What the server sends back: status code, headers and a body.

router

Matching an HTTP method and a URL path to the function that handles it.

server

A program (or the machine running it) that waits for requests and answers them.

service layer

The layer that holds the business rules. It calls repositories and knows nothing about HTTP.

snake_case

Words joined by underscores, like user_name. Common for database columns, not for URIs.

SQL injection

An attack where user input is glued into SQL and changes what the command does. Placeholders prevent it.

summary

Two sizes of DTO: summary has a few fields for lists, detail has everything for one item.